Describing a Voice and Getting It Fast: Qwen3-TTS VoiceDesign at RTF 0.98 on One 8 GB Consumer GPU

Hongyun Du
Independent Researcher
hdu1@my.nl.edu

November 1, 2026

Abstract

Most text-to-speech models can copy a voice from a reference clip; fewer can invent one from a written description. Qwen3-TTS VoiceDesign does the second, and that capability is the reason to consider it, but on a consumer card it ran at RTF 1.41 – slower than playback, so unfit for conversation. We report two settings that bring it to RTF 0.78 with five different descriptions on a single GeForce RTX 3060 Ti, using Hugging Face Transformers and no serving stack. The work is not on the 1.7B talker but on the five-layer code predictor that emits the residual codebooks: about fifteen short forwards per audio frame, each dominated by kernel launch rather than arithmetic. Compiling that predictor alone helps a little; the decisive change is raising `recompile_limit`, which defaults to eight and otherwise falls back to eager silently after the first few shapes. A 20-sentence comparison finds no systematic change in intelligibility or predicted naturalness, with the small differences traceable to the recogniser adding or dropping sentence-initial particles. The cost is a first render of 57–84 s of compilation, which a service must hide behind a readiness gate.

1 Motivation

Most speech models copy a voice from a reference clip: a different timbre means finding or recording another clip. Qwen3-TTS ships three variants – clone only, preset speakers with instructions, and VoiceDesign, which synthesises a speaker from a written description. For a product that wants more than one voice, the third is the interesting one.

What is reported here is only whether that variant can be made fast enough to hold a conversation on one consumer GPU.

2 Where the time goes

Qwen3-TTS [1] is an autoregressive talker followed by a small code predictor. Figures below are read from the released configuration rather than inferred: the talker is 28 transformer layers at width 2048 (1.7B, `tts_model_size: 1b7`); the predictor is five layers at width 1024 with a 2048-entry vocabulary; audio is 16 code groups at 12 Hz (`qwen3_tts_tokenizer_12hz`).

Per audio frame the talker runs once and the predictor runs fifteen times, once per residual group, each on a sequence of two to sixteen tokens. The talker is the large component but the predictor is where the launch count lives – tens of kernels per frame for what is arithmetically a rounding error. Both are worth compiling, for different reasons: the predictor because it is launch-bound, the talker because of its size.

3 Two settings, one of them decisive

Baseline on the 3060 Ti, plain Transformers: RTF 1.41. Profiling one render with the two halves of the pipeline instrumented separately puts 99% of the time in the autoregressive loop and 1% in the vocoder, so the vocoder is not worth touching.

Two compilations inside that loop matter. The code predictor first – a five-layer transformer running about fifteen short forwards per frame – with `mode="default"` and `dynamic=True`: 1.41 to 1.19.

```
torch._dynamo.config.specialize_int = False
torch._dynamo.config.recompile_limit = 64
torch.compiler.set_stance("default")

p = model.model.talker.code_predictor
p.forward = torch.compile(p.forward, mode="default", dynamic=True)

inner = model.model.talker.model          # 1.7B talker, inner transformer
inner.forward = torch.compile(inner.forward, mode="default", dynamic=True)
```

The second compilation is the larger win: 1.19 to 0.78. It has to target `talker.model`, not `talker`: the wrapper's forward calls the predictor's `generate()`, which contains data-dependent control flow, and compiling it raises `UncapturedHigherOrderOpError`. The inner transformer has no such call.

The decisive line is `recompile_limit`. It defaults to eight. The predictor's sequence length grows with every step, so the limit is reached inside the first utterance; torch then emits a single warning and silently falls back to eager for every subsequent shape. Raising it to 64 takes the steady state from 1.19 to 0.98. Anyone measuring only "compilation succeeded" would conclude the optimisation worked, when it had stopped applying after the first few shapes – which is exactly how we first reported it.

`mode="default"` rather than `reduce-overhead`: CUDA graph capture fixes output buffers, and the generation loop reads them across steps, which fails outright. `dynamic=True` is required because the predictor's sequence length changes every frame.

4 Measurements

Configuration	Steady RTF	Peak VRAM	First render
As shipped	1.41	4.09 GiB	—
Compiled, `recompile_limit=8`	1.19	4.06 GiB	75 s
Compiled, `recompile_limit=64`	**0.98**	4.07 GiB	84 s

Table 1: RTX 3060 Ti 8 GB, bf16, no serving stack. RTF is synthesis seconds per audio second; the first render carries compilation and is excluded from the steady figure.

Across five different descriptions – a low-voiced young woman, a bright teenage girl, a composed thirty-something, a whispered voice, and an elderly speaker – the cost is uniform. With both halves compiled, repeated runs on different input lengths measured 0.77, 0.78 and 0.79 on four-second renders, and 0.92 on a seven-second one.

That is not a guarantee of staying below one. A later run of the same recipe, on the same card and with the GPU at normal clocks and temperature, measured 1.03–1.19; a rerun minutes afterwards returned to 0.98. The variable is what else is using the machine. Over all runs the predictor-only configuration sat at 0.97–1.02 and the two-compilation configuration at 0.77–1.19. The second is better on every occasion we measured, and it is not bounded, which is the honest form of the claim.

That margin matters, because the predictor-only configuration sat at 0.97–1.02: right at playback speed, where a busy machine turns a working demo into an occasional stutter. The talker compilation is what buys the headroom.

4.1 Does compiling change the voice?

Twenty Chinese sentences were rendered twice, uncompiled and compiled, and scored with character error rate from a Paraformer pass and UTMOS [3] through its SpeechMOS implementation [4].

Variant	CER (%)	UTMOS
Uncompiled	1.41	3.25
Compiled	2.07	3.23

The 0.62-point CER difference is not a regression. Inspecting the sentences that moved, four changed: in two the compiled variant lost or gained a sentence-initial particle (“ ”, “ ”) that the recogniser had previously caught or missed, and in the other two the uncompiled variant did. Two favouring each side, differing only in magnitude. The recogniser's handling of leading interjections is the variable, not the audio.

5 Cost and consequences

The steady state is now below one, but the first render of a process costs 57–84 s of compilation. A service that reports ready after loading weights will make its first user wait that long, so readiness must be gated on a completed synthesis rather than on weights having loaded. A second process start benefits from the inductor disk cache – 250 s became 100 s – but not to the point where it can be shown to a user.

Peak VRAM is 4.07 GiB, which fits a 6 GB card with room for a desktop and an 8 GB card comfortably. Weight quantisation was not attempted: at this footprint it would buy nothing on the target card.

6 What this does not establish

- One machine, one model revision, single runs. The third digit is noise. - No listening test. UTMOS [3] is trained to predict mean opinion scores from listening tests on other data; it is not a listener, and its target is naturalness rather than whether a voice matches the description it was asked for. That is a human judgement and we have not made it systematically. - Speaker similarity is not scored. VoiceDesign has no reference clip; the description is the specification, so there is nothing to compute a cosine against. Whether ten different descriptions produce ten genuinely different speakers is a question for a listener. - Only the code predictor was compiled. The talker was left eager, and CUDA graphs were not pursued.

7 Reproduction

A script accompanies this note: it loads the VoiceDesign checkpoint, applies the three settings, and renders five described voices, reporting steady RTF and peak memory while writing the audio.

References

[1] Qwen Team. *Qwen3-TTS-12Hz-1.7B-VoiceDesign* model card. `https://huggingface.co/Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign`.

[2] PyTorch Team. *torch.compile* and *torch.dynamo* documentation, including `recompile_limit` and configuration semantics.

[3] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, H. Saruwatari. *UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022*. INTERSPEECH 2022; arXiv:2204.02152.

[4] T. Saeki. *SpeechMOS*: MOS prediction toolkit implementing UTMOS. `https://github.com/tarepan/SpeechMOS`.